Ten phases to a 10/10 website

A practical roadmap from the repository audit to a secure, reliable and polished multi-tenant commerce platform.

01 Security

02 Identity & tenants

03 Orders & stock

04 Pricing & payments

05 Backend architecture

06 Reliable jobs

07 User experience

08 Speed & scale

09 Tests & visibility

10 Production operations

Each phase includes the problem, ordered work, owner, effort, dependencies, acceptance criteria and rollout considerations.

PREPARED 27 SEPTEMBER 2026

Audit baseline: commit 6b37177 | Planning document, not a release certification

What 10/10 means here

A measurable quality bar for your existing product, supported by working software and operating evidence.

This plan covers the Next.js storefront, React admin, Express API, MongoDB, background worker and shared storefront packages. It builds on the completed repository audit and the critical-dependency follow-up. No application changes are made by this document.

What is established, and what still needs proof

Area	Audit evidence	Planning consequence
Security	Affected Next.js and Multer versions are in the lockfiles; actual deployed exploitation was not demonstrated.	Patch and verify the running artifacts first.
Commerce	State-transition, return, idempotency and price-consistency defects were traced in code.	Treat money and stock invariants as release blockers.
Verification	Backend: 289/290 tests passed; one CRLF-sensitive assertion failed. Admin/storefront lint and builds passed.	Preserve existing checks; add runtime integration and concurrency tests.
Production	No verified production topology, load profile, restore result or database state. Docker was unavailable for integration tests.	Collect deployment evidence and baseline metrics; do not infer capacity from code.

The scoring rule

Each of the ten phase areas earns 0 points when incomplete or unverified, 0.5 when implemented and verified in staging, and 1 when its acceptance evidence is complete and the production observation criteria pass. Add the ten values for the internal score. Current values are **not reassessed** in this document.

Hard gates override the total

A high average never offsets an unresolved applicable critical/high vulnerability, a tenant-data leak, an order/payment integrity defect, or a failed recovery drill. All ten areas must earn full credit to claim 10/10 against this agreed scope.

Evidence labels: **E01-E20** identify repository findings on pages 17-18. **S01-S09** identify primary references on page 19. API latency, availability, recovery and effort values are proposed project targets; Core Web Vitals thresholds are externally published.

Sequence the work by risk

Engineering effort is an initial planning estimate. Re-estimate after Phase 1 with the actual team, providers and deployment environment.

Phase	Primary outcome	Depends on	Effort
01	Close critical security exposure	Start now	4-6 days
02	Prove identity and tenant isolation	Phase 1	6-10 days
03	Protect orders, stock and returns	Phase 2	10-15 days
04	Unify prices, payments and billing	Phase 3	8-12 days
05	Make the backend easy to change	Phases 2-4	8-12 days
06	Make background work recoverable	Phases 3 and 5	6-10 days
07	Deliver a clear, accessible storefront	Phase 4 contracts; design can start earlier	10-15 days
08	Meet measured speed and scale targets	Phases 5-7; measure from Phase 1	6-10 days
09	Make quality and failures visible	Incremental throughout; certify Phases 1-8	10-15 days
10	Launch safely and keep improving	All prior release gates	6-10 days

Total: 74-115 engineer-days, excluding provider onboarding, procurement and the final 30-day observation window. A provisional range is 10-16 calendar weeks with two engineers plus part-time QA, design and platform/security review; sequencing and interruptions can extend it. This is not a delivery commitment.

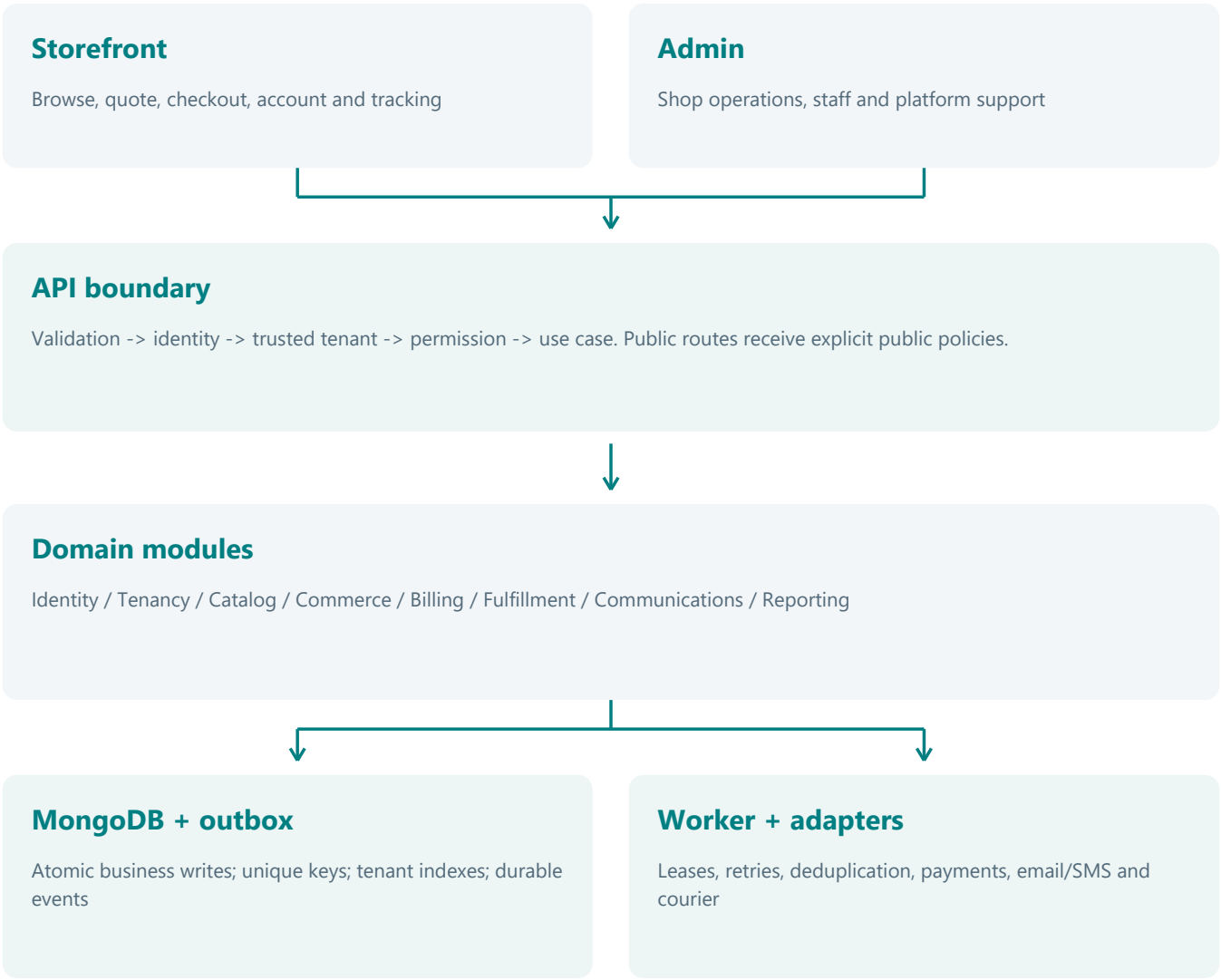
Start with three concrete tickets

1. Upgrade and verify Next.js/Multer plus restrict image sources.
2. Fix return aggregation and cumulative refund/stock limits.
3. Unify checkout totals through a server quote and regression scenarios.

Work that starts early: test fixes with each change; collect latency/error baselines during Phase 1; start UX research while backend contracts settle; prepare staging and restore procedures before the final launch phase. Phase 9 consolidates verification, and Phase 10 certifies operations.

Keep a modular monolith

Strengthen the existing boundaries and operating controls, then scale the specific bottlenecks that measurements reveal.



Shared operational services

Use bounded, tenant-aware caching; scoped Cloudinary media access; structured logs, traces and metrics; immutable deployment artifacts; secrets management; and verified backups. The API and worker can scale independently while sharing the same domain contracts.

Migration discipline

Move one business flow at a time behind a stable interface. Add constraints only after reconciling existing records. Keep provider-specific code in adapters and cross-module events explicit. Extract an independent service only when ownership, failure isolation or measured load justifies it.

PHASE 01 / 10

Close critical security exposure

Owner: Backend + frontend engineers; security reviewer
Estimate: 4-6 engineer-days | **Dependency:** Start now

The storefront pins Next.js 16.2.10; the backend pins Multer 2.2.0. Published advisories cover remote code execution in specific Next.js deployments and a Multer process crash. The image allowlist trusts every Cloudinary account. Production exploitability remains unverified. [E01, E02; S01-S04]

01 **Patch the runtime dependencies.** Upgrade Next.js to at least 16.3.3 and Multer to at least 2.3.0, or later compatible releases verified against current advisories. Align Next lint tooling, update lockfiles and examine every production dependency alert.

02 **Constrain the image and upload surfaces.** Restrict Cloudinary optimization to the configured cloud account and approved paths. Authorize return evidence uploads before expensive storage work; use a separate upload token when multipart fields currently carry authorization.

03 **Make identity recovery revoke access.** Increment the live session version atomically on password recovery and security-sensitive password changes. Define current-device and all-device logout. Remove legacy token acceptance through a tested migration. [E03]

04 **Verify operating conditions.** Record the deployed Next version, hosting OS, optimizer ownership and exposed upload routes. Separate readiness from liveness immediately; return a failure status when required MongoDB access is unavailable. [E13]

05 **Create the first regression gate.** Capture safe upload rejection, image allowlist, session revocation and dependency checks in CI. Replace raw production 500 messages with a stable error code and request ID. [E03]

Exit criteria - required evidence

Patched artifacts are confirmed in the running deployment; no unresolved applicable critical/high security finding remains. Old sessions fail after recovery; other Cloudinary accounts are rejected; malformed multipart requests cannot terminate the isolated test server.

Delivery and rollout: Deliver dependency diffs, exposure notes and a security regression report. Canary the new build. Keep a patched rollback image; rolling back to vulnerable packages is not an acceptable recovery path.

PHASE 02 / 10

Prove identity and tenant isolation

Owner: Backend engineer; security + QA reviewers
Estimate: 6-10 engineer-days | **Dependency:** Phase 1

Reviewed routes use tenant context and membership checks; no cross-tenant exploit was confirmed. The remaining work is systematic proof across every mounted route, background job, cache key and stored object. Password revocation belongs to the first security release. [E03, E04]

01 **Publish an access matrix.** List customer, shop owner, staff and platform roles against every resource action. Document intentional platform-wide access separately, with reason capture and audit records for sensitive support operations.

02 **Make tenant context mandatory.** Resolve the shop from a validated domain or authenticated membership. Require trusted tenant context in domain services and repository methods. Reject client-supplied ownership fields; validate referenced products, orders and files within the same shop.

03 **Test the complete boundary.** Use two shops and two users with deliberately similar objects. Exercise read, update, delete, export, search, bulk actions and upload IDs. Cover removed membership, suspended shops, staff permission changes and custom-domain reassignment.

04 **Strengthen privileged identity.** Introduce MFA for platform operators and sensitive shop roles, recovery procedures and reauthentication for high-impact actions. Preserve cookie protections and verify CSRF/origin behavior against the actual domain topology.

05 **Define personal-data handling.** Turn privacy requests into a documented workflow with identity verification, export/erasure scope, retention exceptions and completion evidence. Set access and retention rules for identity documents, order data and support attachments. [E18]

Exit criteria - required evidence

Every mounted protected action has a policy owner and positive/negative tests. All cross-user and cross-shop attempts fail without revealing private data. Revoked access takes effect immediately; privileged actions produce redacted audit events. [S05]

Delivery and rollout: Deliver the access matrix, tenant test fixtures and privacy workflow. Roll out policy changes by route family and observe denied requests. Resolve legacy membership data before enforcing stricter invariants.

PHASE 03 / 10

Protect orders, stock and returns

Owner: Backend engineer; commerce owner + QA
Estimate: 10-15 engineer-days | **Dependency:** Phase 2

Order status updates can permit invalid backward transitions. Duplicate return lines can inflate quantities and refund amounts, while partial refunds can mark the whole order refunded. Inventory validation failures can leave idempotency keys stuck in processing. [E05-E07]

01 **Approve explicit state machines.** Define allowed order and return transitions, actors, conditions and side effects. Keep fulfillment, payment and refund states separate. Reject stale updates using an expected current state or version in the write filter.

02 **Make each commerce mutation atomic.** Apply order, inventory, return balance and audit/outbox changes within a MongoDB transaction where they span documents. Use conditional stock decrements, unique idempotency constraints and integer quantity validation. [S06]

03 **Normalize return requests.** Aggregate duplicate product/variant/order-line IDs before validation. Compare requested quantities with purchased minus already returned/reserved quantities. Cap cumulative refunds at the refundable paid amount and model partial refunds explicitly.

04 **Repair retry semantics.** Validate before reserving an idempotency key where possible. Bind the key to actor, tenant, operation and request hash; persist the result. Define processing leases, failed states and safe recovery after a crash.

05 **Reconcile existing data.** Produce a dry-run report of invalid transitions, duplicate returns, stale reservations and stock discrepancies. Review affected records before correction. Preserve an audit trail and require reconciliation after the migration.

Exit criteria - required evidence

With stock of one, simultaneous purchases produce one successful allocation. Replaying an identical request returns the same result; a changed payload with the same key fails. Duplicate or concurrent returns cannot over-refund or over-restock. Interrupted transactions leave no partial commerce mutation.

Delivery and rollout: Deliver state diagrams, transaction services, reconciliation scripts and concurrency tests. Protect the rollout with a backup and an additive schema migration; use compensating business events for completed financial actions.

PHASE 04 / 10

Unify prices, payments and billing

Owner: Backend + frontend engineers; finance/product owner
Estimate: 8-12 engineer-days | **Dependency:** Phase 3

Outside-Dhaka shipping is shown as BDT 120 but charged as BDT 130. Scheduled-sale cart prices and coupon previews can drift. FIRST_ORDER eligibility checks same-coupon usage. Manual billing approval does not enforce invoice amount equality; online payment completion is not established. [E08-E10]

01 **Create a server-authoritative quote.** Return line prices, discounts, shipping, fees, currency, total and quote expiry/version from one pricing service. Use integer minor units or a tested decimal type. Render the same breakdown in product/cart/checkout where relevant.

02 **Revalidate changes explicitly.** Refresh quotes when quantity, variant, address, customer or coupon changes. Clear stale coupon previews. At order submission, recompute and require user acceptance when the payable amount changes.

03 **Correct promotions and billing.** Check first-order eligibility against qualifying historical orders with an explicit cancellation policy. Compare manual payment amount/currency to the invoice; show expected and submitted values together. Make approval and entitlement activation idempotent.

04 **Implement only supported payment journeys.** Keep COD clearly labeled. Enable bKash, Nagad or card checkout only after verified provider callbacks, amount/currency checks, unique transaction IDs, pending/failed/refunded states and reconciliation are implemented against provider documentation.

05 **Verify totals and business reporting.** Enforce order-total invariants before persistence. Reconcile orders, payments, refunds and invoices daily. Correct the analytics rollup that counts item rows as orders before any dashboard consumes it. [E19]

Exit criteria - required evidence

Every shipping zone, sale, variant and coupon scenario produces identical displayed and persisted totals. Old customers cannot reuse first-order benefits. Underpaid or mismatched invoices cannot activate a full plan. Duplicate provider events never double-charge or double-credit.

Delivery and rollout: Deliver pricing contracts, payment state diagrams and a finance reconciliation report. Use provider sandboxes, then a limited live pilot. Provider onboarding time is outside the estimate; leave unsupported methods disabled until verified.

PHASE 05 / 10

Make the backend easy to change

Owner: Principal/backend engineer; API consumers
Estimate: 8-12 engineer-days | **Dependency:** Phases 2-4

The code already contains services, but several controllers still combine validation, database work and domain rules. Route reachability and error contracts need consistency. Purchase-order routes exist without being mounted. [E05, E07, E12]

01 **Evolve a modular monolith.** Create explicit identity, tenancy, catalog, commerce, billing, fulfillment, communications and reporting modules. Keep controllers thin: parse input, authorize, invoke a use case and serialize a response.

02 **Centralize domain rules.** Move quote calculation, state transitions, stock mutations and entitlement changes behind one service boundary per use case. Require tenant context in data access. Keep public response schemas separate from database documents.

03 **Make API contracts executable.** Document request/response schemas, pagination, filters, errors and idempotency behavior. Validate input at entry points and return safe errors with stable codes, field details and request IDs.

04 **Resolve incomplete surfaces.** Either mount purchase-order APIs with the full authorization and test chain, or explicitly defer and remove dead entry points. Paginate customer order history and constrain list sizes and sort fields. [E12, E16]

05 **Extract shared contracts carefully.** Share pure pricing types, status names and renderer interfaces where useful; keep server authority on the server. Add architecture checks for prohibited cross-module imports and document ownership in short decision records.

Exit criteria - required evidence

Every mounted endpoint is in the API inventory and has a schema plus access policy. A state-transition or pricing rule has one authoritative implementation. Two representative route families use the new pattern with unchanged public contracts and passing regression tests.

Delivery and rollout: Deliver the module map, API specification and incremental extraction plan. Migrate one vertical flow at a time. Avoid a repository-wide rewrite; preserve compatibility during the transition and remove adapters only after consumers migrate.

PHASE 06 / 10

Make background work recoverable

Owner: Backend/platform engineer; integration owners
Estimate: 6-10 engineer-days | **Dependency:** Phases 3 and 5

Several scheduled maintenance tasks run only when the job queue is idle. Busy queues can therefore delay sales and cleanup. Five-minute job reclaim has no lease renewal, and campaign retries can resend to earlier recipients. [E11]

01 **Separate scheduling from job consumption.** Run periodic scheduling on its own clock and persist a deduplicated job per time window. Ensure sales activation, outbox dispatch and cleanup remain timely while the normal queue is continuously busy.

02 **Add leases and ownership.** Use atomic claims, worker identity, heartbeat renewal and a fencing/version check on completion. Expired workers must not commit results after another worker has reclaimed the job. Set timeouts and bounded exponential retry with jitter.

03 **Make side effects replay-safe.** Write outbox events with the originating commerce transaction. Track delivery attempts and completion. Persist per-recipient campaign progress and pass stable idempotency keys to providers that support them.

04 **Handle ambiguous provider outcomes.** Define timeouts, rate limits, dead-letter handling and reconciliation for email/SMS, media, courier and payment services. Where a provider cannot deduplicate, query delivery status or require review before retrying an uncertain send.

05 **Expose operational controls.** Add queue age, attempts, failure reason and redacted correlation IDs. Provide an authorized inspect/retry/cancel workflow. Correct subscription-plan updates separately as controlled migrations in Phase 10. [E13]

Exit criteria - required evidence

A sustained backlog does not stop due schedules. Killing a worker during a campaign does not repeat completed recipients where delivery can be verified. Reclaiming an expired lease prevents the former owner from committing. Poison jobs are isolated and visible to operators.

Delivery and rollout: Deliver a job contract, outbox processor and recovery runbook. Drain old workers before changing lease formats. Treat execution as at-least-once; prove idempotent business effects instead of claiming universal exactly-once delivery.

PHASE 07 / 10

Deliver a clear, accessible storefront

Owner: Frontend engineer; product designer + QA
Estimate: 10-15 engineer-days | **Dependency:** Phase 4 contracts; design can start earlier

The audit found an OTP countdown/state conflict, missing form-label associations, incomplete modal semantics, zero-stock quantity fallbacks and role redirect errors. These create avoidable friction in account, cart and admin journeys. Visual redesign needs user validation. [E14, E15]

01 **Define a coherent visual system.** Standardize typography, spacing, color, focus states, forms and buttons across storefront templates. Preserve tenant branding within accessible limits. Test mobile layouts at 360px, tablet and desktop widths using representative long content.

02 **Repair critical interaction states.** Unify OTP state and timer recovery; associate labels and field errors. Give dialogs focus management, keyboard dismissal and accessible names. Correct role-aware redirects and clearly distinguish expired sessions from forbidden actions.

03 **Make buying predictable.** Fix zero-stock handling for variants and cap quantities from server availability. Show a complete quote before confirmation, retain recoverable cart data, prevent duplicate submission and provide useful pending, retry, empty and offline states.

04 **Build trust and findability.** Make delivery costs, return policy and seller contact easy to find. Audit product metadata, structured data, sitemap inclusion, canonical domains, redirects and private-page indexing. Verify custom-domain certificate and ownership journeys.

05 **Validate with real users.** Review WCAG 2.2 AA with automated checks plus keyboard and screen-reader testing. Observe at least five representative shoppers and three vendors on scripted tasks; fix every blocker and repeat failed journeys. [S07]

Exit criteria - required evidence

A user can register, buy, track and request a return on mobile and keyboard without a blocking defect. No critical/serious automated accessibility finding remains on core pages; manual review covers the remaining criteria. Canonical and structured data match actual shop and product content.

Delivery and rollout: Deliver reusable components, journey recordings and an accessibility issue log. Roll out by storefront template, preserve cart compatibility and use feedback from the pilot to prioritize the next refinement.

PHASE 08 / 10

Meet measured speed and scale targets

Owner: Frontend + backend engineers; platform reviewer
Estimate: 6-10 engineer-days | **Dependency:** Phases 5-7; measure from Phase 1

Customer orders are unbounded, short order-number lookup uses a computed expression, cache fallback lacks a firm size bound, and query-shaped cache keys can grow rapidly. Actual production throughput and data volume were not available. [E16, E17]

01 **Agree on the workload.** Record catalog size, active shops, peak browse/checkout rates, geographic mix and hot-tenant behavior. Build a representative dataset and test at the forecast peak plus 2x headroom. User count alone does not establish capacity.

02 **Optimize database access.** Use explain plans for slow paths, add justified tenant-first indexes and bounded pagination, and replace computed short-ID lookup with an indexed canonical field after a collision/backfill review. Project only needed fields.

03 **Bound and isolate caching.** Set memory limits, eviction and expiry for local fallback; normalize cache keys and include tenant identity. Coordinate invalidation across API instances. Classify which routes can safely degrade when Redis is unavailable.

04 **Improve the browser experience.** Measure bundle cost, server waterfalls and image bytes. Prioritize the actual largest image, reserve media dimensions, defer noncritical UI and keep checkout dependencies small. Verify CDN and private-response cache boundaries.

05 **Protect fairness and cost.** Set per-tenant and global limits for expensive exports, analytics, uploads and jobs. Load-test skewed traffic from one shop and track database connections, memory, queue lag and cost per order.

Exit criteria - required evidence

Proposed targets: p75 LCP <=2.5s, INP <=200ms and CLS <=0.1 on mobile and desktop [S08]. At the agreed peak, p95 catalog API <=400ms, internal order creation <=800ms and unexpected errors <0.5%; report provider time separately. Confirm bounded memory during a one-hour soak.

Delivery and rollout: Deliver before/after traces, query plans, load results and a capacity envelope. These API targets are project proposals, not measured current performance. Use lab budgets until sufficient field samples exist; document cache warm-up and invalidation during rollout.

PHASE 09 / 10

Make quality and failures visible

Owner: QA + engineers; reliability owner
Estimate: 10-15 engineer-days | **Dependency:** Incremental throughout; certify Phases 1-8

The earlier audit ran 290 backend tests: 289 passed and one source-regex assertion failed on CRLF line endings. Both frontends passed lint/build. Mongo integration tests could not run because Docker was unavailable. These results leave concurrency and end-to-end behavior unproven. [E20]

01 **Use realistic test infrastructure.** Run integration tests against an ephemeral MongoDB replica set so transactions behave correctly. Seed two tenants, multiple roles, products, orders and returns. Replace brittle source-text assertions with observable behavior where practical.

02 **Gate the critical journeys.** Add browser tests for guest/member checkout, OTP, sale/coupon updates, stock contention, order tracking, returns, billing approval, staff restrictions and custom-domain navigation. Preserve regression cases from every audit fix.

03 **Exercise failure and recovery.** Inject worker death, duplicate provider callbacks, slow dependencies, DB disconnects, Redis loss and upload rejection. Verify business invariants and user feedback after each failure. Quarantine flaky tests only with an owner and expiry.

04 **Connect logs, metrics and traces.** Carry request, tenant-safe correlation, order and job identifiers across boundaries. Instrument API latency, order failures, quote mismatch, queue lag, refunds and readiness. Redact credentials, OTPs, identity documents and personal data. [S09]

05 **Make alerts actionable.** Choose a proposed 99.9% monthly availability objective for critical journeys, define the measurement boundary and alert on error-budget burn. Route incidents to an owner; run synthetic checkout tests in a dedicated test tenant.

Exit criteria - required evidence

CI runs unit, replica-set integration, API contract, tenant isolation and core browser tests. Three consecutive clean runs show no unexplained flaky critical test. A simulated checkout incident triggers an alert with a request trace and usable runbook; secrets stay out of telemetry.

Delivery and rollout: Deliver CI artifacts, dashboards, alert rules and incident drills. Existing tests remain the starting point. Passing build/lint or achieving a code-coverage percentage alone does not close this phase.

PHASE 10 / 10

Launch safely and keep improving

Owner: Platform engineer; product + security owners
Estimate: 6-10 engineer-days | **Dependency:** All prior release gates

Actual backups, production indexes, deployment controls and incident response were not evidenced. API and worker startup both synchronize subscription plans, which can change entitlements during deployment or rollback. [E13] A production-quality system needs repeatable operations.

01 **Make deployments reproducible.** Separate development, staging and production credentials and data. Validate required environment variables on startup. Build immutable artifacts, record dependency versions and deploy API, worker and storefront through a documented pipeline.

02 **Control migrations and releases.** Move subscription-plan synchronization into a versioned, reviewed migration with dry-run output and audit records. Use additive schema changes and compatible releases. Deploy a canary and verify business metrics before expanding traffic.

03 **Prove disaster recovery.** Configure encrypted database backups and point-in-time recovery where available. Set proposed RPO <=15 minutes and RTO <=60 minutes, then restore into an isolated environment and reconcile orders, payments and inventory to prove them.

04 **Complete the operating model.** Assign on-call ownership, incident severity, vendor escalation and privacy request handling. Add secret rotation procedures, certificate/domain monitoring, retention schedules and a monthly vulnerability review with urgent patch handling.

05 **Run a controlled launch and learn.** Pilot with a small set of shops, gather shopper and vendor feedback, inspect refunds/support burden and monitor conversion without counting untrusted browser events as revenue. Expand only while hard gates and service objectives hold.

Exit criteria - required evidence

A rollback, restore and incident drill all succeed with retained evidence. The running deployment matches the reviewed artifact; readiness removes unhealthy instances. Every scorecard dimension passes, then remains within agreed targets through a 30-day observation window.

Delivery and rollout: Deliver the release evidence pack, environment inventory, recovery report and named operating owners. The effort estimate excludes the 30-day observation period. Treat the 10/10 assessment as renewable evidence after significant changes.

Tests that protect the business

Build these cases as the related changes land. Use real database transactions and provider sandboxes where the behavior depends on them.

Scenario	Expected result	Evidence / phase
Account recovery	Previously issued sessions cannot use protected APIs after recovery commits.	API + live account state / 1-2
Two tenants, same roles	Cross-shop reads, writes, exports, media and search attempts fail without private data.	Policy matrix + API suite / 2
Two buyers, one unit	One successful allocation, one clear stock response, no negative stock or orphan order.	Replica-set race test / 3
Checkout retry	An identical request/key returns the same order; a changed payload is rejected.	API + DB assertions / 3
Partial return replay	Duplicate lines and concurrent returns never exceed remaining quantity or refundable balance.	Transaction test / 3
Quote changes	Address, sale expiry, cart or coupon changes update the shown total and persisted order consistently.	API + browser suite / 4
Payment and invoice approval	Duplicated callbacks are harmless; wrong amount/currency cannot mark an invoice paid.	Provider sandbox + DB / 4
Worker death and backlog	Due tasks still run; reclaimed jobs cannot commit from stale workers; retries preserve progress.	Failure injection / 6
Accessible mobile purchase	Keyboard/screen reader and small-screen flows complete with clear labels, focus and errors.	Manual review + automation / 7
Tenant traffic spike	One busy shop cannot exhaust shared resources; latency and memory stay within the agreed envelope.	Load/soak report / 8
DB loss and production restore	Readiness fails, alerts fire, traffic drains and a restored copy reconciles business records.	Incident + restore drill / 9-10

For every result, retain the commit/build identifier, environment, test data description, expected result, actual result and reviewer. A screenshot alone does not prove transaction or authorization correctness.

The 10-point release scorecard

Use the same evidence standard for each area. An unchecked row contributes no claim of production readiness.

Point	Full-credit requirement	Acceptance owner
1. Security	Verified patched deployment; applicable critical/high findings closed; recovery revokes sessions.	Security reviewer
2. Isolation	Complete policy matrix and negative tenant tests; privileged access and privacy workflow verified.	Backend + QA
3. Integrity	Orders, stock, returns and retries preserve invariants under races and failures.	Commerce owner
4. Money	Quote consistency; correct promotion eligibility; payment/refund and invoice reconciliation.	Finance + product
5. Architecture	Clear module ownership; complete API contracts; domain rules have one authoritative path.	Technical lead
6. Background work	Schedules survive backlog; lease recovery, deduplication and operational retries are proven.	Platform owner
7. Experience	Core mobile/accessible journeys pass; trust content and SEO/domain behavior are verified.	Design + QA
8. Performance	Measured traffic envelope; performance budgets met; bounded memory and tenant fairness.	Engineering lead
9. Quality and visibility	Critical CI suite is dependable; alerts, redaction and traces work in incident drills.	QA + reliability
10. Operations	Canary, rollback and restore succeed; owners and runbooks exist; observation targets hold.	Release owner

Release evidence pack

Store the dependency report, access matrix, API spec, state diagrams, migration dry runs, CI results, accessibility review, load report, reconciliation report, alert drill and restore report with the release identifier.

Renewal: reassess after significant architecture, payment, authentication or hosting changes. Keep a monthly security and reliability review. A 10/10 score represents the agreed scope and evidence window; it cannot guarantee the absence of future defects.

Code evidence index 1/2

Locations refer to audit baseline 6b37177. Recheck line numbers after edits; findings describe the audit, not a new full scan.

E01 - Next dependency and image surface

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-storefront/package-lock.json:5100

Next 16.2.10; also next.config.mjs:35 and src/lib/imageDomains.js:17. Rechecked in the critical-security follow-up.

E02 - Multipart dependency and public return upload

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/package-lock.json:1476

Multer 2.2.0 is locked; routes/storefrontRoutes.js:250 defines the public return upload. Parsing precedes order-access checks. Verify the deployed artifact during Phase 1.

E03 - Sessions and error handling

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/controllers/authController.js:1022

Password update without session-version bump; also services/passwordResetService.js:561, middlewares/auth.js:20 and middlewares/error.js:18.

E04 - Live authorization and legacy token paths

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/middlewares/auth.js:20

Live account/membership/version checks; legacy accountless branch around line 91. No confirmed cross-tenant exploit in reviewed routes.

E05 - Order and return state transitions

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/controllers/orderController.js:584

Unrestricted nonterminal order transitions; also services/orders/orderStatusService.js:1 and controllers/returnController.js:22.

E06 - Return quantities and refund effects

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/controllers/publicController.js:1231

Per-entry return checks allow duplicate lines; also controllers/returnController.js:43 and :378 for restock/refund effects.

E07 - Inventory reservations and validation

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/controllers/inventory.js:65

Idempotency reservation can remain processing after early validation errors; quantity validation around line 39.

E08 - Shipping, sale and coupon display drift

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-storefront/src/app/[subdomain]/checkout/hooks/useCheckoutTotals.js:14

BDT 120 vs backend orderPricingService.js:4 at BDT 130; sale cart recomputation in src/hooks/useStorefrontProductActions.js:40; stale coupon preview in useCheckoutCoupon.js:28.

E09 - First-order eligibility

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/services/promotionService.js:91

Order-use count scoped to the same coupon affects FIRST_ORDER eligibility; enforcement also around line 209.

E10 - Manual billing and payment completion

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/services/billing/paymentVerificationService.js:137

Approval lacks amount-equality invariant; submission around line 29. Order payment enums exist, but online capture/verification was not established by audit.

Code evidence index 2/2

Locations refer to audit baseline 6b37177. Recheck line numbers after edits; findings describe the audit, not a new full scan.

E11 - Worker starvation and repeat effects

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/workers/index.js:175

Maintenance depends on queue idle; also services/jobQueueService.js:57 for reclaim and customerEmailCampaignService.js:151 for recipient progress.

E12 - Unmounted purchase-order API

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/routes/purchaseOrderRoutes.js:14

Routes defined but absent from app.js route mounts. Decide whether to complete or defer this feature.

E13 - Readiness and startup plan synchronization

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/app.js:137

Health can return 200 while Mongo is disconnected; app startup sync at :180 and worker sync at :153. Plan script can reset feature matrices on version mismatch.

E14 - OTP, labels and modal interaction

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-storefront/src/app/[subdomain]/account/page.jsx:31

Countdown restoration conflicts with OTP state reset; form semantics in components/account/AuthView.jsx:37. Manual accessibility validation remains necessary.

E15 - Zero-stock cart and admin redirect

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-storefront/src/app/[subdomain]/cart/page.jsx:227

Truthy fallback can replace zero variant stock; role redirect mismatch in ecommerce-admin/src/App.jsx:64 and :192.

E16 - Unbounded history and computed lookup

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/services/orders/orderQueryService.js:11

Customer order history lacks a bound; publicController.js:98 uses a computed short order-ID suffix query.

E17 - Cache growth and invalidation risk

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/services/cacheService.js:1

Process-local fallback has no size bound; storeController.js:225 constructs query-dependent bootstrap cache keys.

E18 - Privacy request workflow

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/controllers/privacyController.js:1

Deletion creates a DataRequest workflow; automatic erasure was not evidenced. Define operational completion and retention rules.

E19 - Analytics integrity

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/scripts/runAnalyticsRollup.js:46

Unwound items are counted as orders in a latent rollup; no current reader found. Public analyticsEventController.js:52 accepts client-selected object IDs.

E20 - Test limitations and CI evidence

C:/Users/user/WebstormProjects/Shop-for-all/ecommerce-backend/backend/tests/security-hardening.test.js:1029

Hardcoded newline source assertion failed on CRLF; fields were present. Frontend lint/build passed; transaction integration tests remained unrun in the audit.

Sources and implementation handover

Reference material checked on 27 September 2026. Recheck advisories and provider requirements when implementation begins.

S01 - Next.js AVIF optimization advisory

Critical dependency issue; patched Next.js 16.x baseline is 16.3.3. Runtime exposure depends on the actual image path.
https://github.com/vercel/next.js/security/advisories/GHSA-2xp9-vwfh-vxw4

S02 - Next.js Windows-hosted server advisory

Critical Windows-filesystem condition; the local Windows checkout does not establish production hosting.
https://github.com/vercel/next.js/security/advisories/GHSA-p293-qw3h-jr36

S03 - Multer multipart field-name advisory

High-severity unauthenticated process crash; versions below 2.3.0 are affected.
https://github.com/advisories/GHSA-wc9g-mqfw-jrwm

S04 - Next.js image remotePatterns

Use explicit host/account paths; omitted pathname patterns are broad.
https://nextjs.org/docs/app/api-reference/components/image#remotepatterns

S05 - OWASP Authorization Cheat Sheet

Least privilege, default denial and permission validation on every request.
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

S06 - MongoDB atomicity and transactions

Reference for conditional writes and multi-document transaction boundaries. Consulted in the earlier audit; direct fetch failed during PDF preparation.
https://www.mongodb.com/docs/manual/core/write-operations-atomicity/

S07 - W3C Web Content Accessibility Guidelines 2.2

Use AA as the target, with automated and manual checks across complete user journeys.
https://www.w3.org/TR/WCAG22/

S08 - Google Web Vitals

Good thresholds: LCP 2.5s, INP 200ms and CLS 0.1 at the 75th percentile, segmented by device.
https://web.dev/articles/vitals

S09 - OpenTelemetry documentation

Vendor-neutral instrumentation for traces, metrics and logs; select a backend that fits the operating budget.
https://opentelemetry.io/docs/

How to use this document

Create a ticket for each numbered work item, name its owner and attach the phase exit criteria. Complete the first security release, then re-estimate the remaining plan using real deployment, data and provider evidence. Review the scorecard with the product owner before launch.